Lab Title: Lab Title: Investigating Memory Evidence

Student Name: Samuel Chinedu Onu

Student ID: 2025/FWSD/11518

Course Name: SBT-DF204: Computer Forensics Case Studies

Instructor Name: Aminu Ibrahim Jalo

Date of Submission: 18th October 2026

Computer Forensics Case Studies

Case Study 3 Investigating Memory Evidence

Student Assessment Brief

Assessment detail	Information
Course	SBT-DF204 Computer Forensics Case Studies
Assessment	Case Study 3 Individual memory forensics investigation
Case title	Investigating Suspected Illegal File Transfer From Memory Evidence
Submission format	Written report with Volatility output, screenshots, timeline, and evidence log
Deadline and maximum score	As published in E-Campus

This individual case study asks you to examine a Windows 7 memory image and reconstruct an evidence-based account of a suspected illegal file transfer. Use Volatility 2 to identify relevant processes, network artefacts, command history, removable-media records, user-profile evidence, and File Explorer traces. Your report must clearly separate direct observations from inferences about a transfer or the person who used the workstation.

Case scenario

Investigators obtained a Windows 10 memory image from a workstation after a confidential document may have been transferred without authorization. The inquiry is whether browser activity, a command prompt, a removable device, and a user profile form a defensible sequence. This is an offline examination of the supplied material. It does not authorize contact with any address, host, account, file-sharing service, or person that may appear in the image.

Case materials

The supplied case file is a Windows 7 memory image named memdumpWin7.mem. The teaching deck demonstrates Volatility 2 methods. In the ICDFA Kali lab, start Volatility 2 with python2 vol.py. Create a working copy before analysis. Record the source, original file name, file size, SHA-256 value, and tool version. Select and justify the memory profile from the actual image output instead of assuming one from a screenshot.

What you must submit

Item	Required output	What to provide
1	Forensic working copy	Acquire the supplied memory image. Record its source, original file name, byte size, SHA-256 value, and the location of your read-only original and working copy.
2	Volatility evidence set	Preserve output from profile identification, process and network analysis, command history, USB registry and MountPoints2 checks, and ShellBag analysis. Label every output with the plugin, profile, and command used.
3	Reconstructed timeline	Build a time-ordered account of the suspected transfer. Cite process IDs, registry paths, offsets or record details, timestamps, and evidence-log IDs where relevant.
4	Student report	Submit one report in PDF or DOCX format with labelled screenshots, a completed evidence log, and a timeline appendix.

Investigation task

Examine the memory image methodically. Base every finding on identifiable Volatility output, including the plugin, profile, command, timestamp, record, registry path, process ID, or offset that supports it. A recovered string, process name, command, or registry key can be important, but it may not prove that a transfer completed or that a particular person performed it. Use the questions below to structure your report.

No	Investigation question	Minimum evidence to discuss
1	What evidence image did you acquire and how did you preserve it?	Record the source, file name, file size, SHA-256 value, working-copy method, file-access precautions, and tools used.
2	What operating-system profile and image time are supported?	Provide imageinfo output, the candidate and selected profile, the image time, and the time zone. Explain how you validated the profile.
3	What user and account artefacts are relevant?	Identify the account or profile records examined, their SIDs or registry-hive paths, and the limit between profile evidence and proof of a person at the keyboard.
4	What process and network evidence relates to a potential download or transfer?	Identify relevant process IDs, process names, local and foreign endpoints, connection state, and memory strings or output records. Do not contact any endpoint.
5	What does command history show?	Use cmdscan or consoles output to identify relevant commands, the command process, path or drive references, and the timing or correlation evidence. Explain any limit on proving success.
6	What removable-media evidence is present?	Use USBSTOR, DeviceClasses, MountedDevices, and MountPoints2 evidence as available to identify device, drive-letter, and user-profile relationships. State what each artefact does and does not establish.
7	What File Explorer evidence supports the sequence?	Use ShellBags to identify material folder paths and recorded access or preference times. Distinguish folder-view evidence from a file-modification timestamp.
8	What conclusion can you defend?	Create a UTC timeline and state a cautious conclusion with a confidence level and at least three limitations or alternative explanations.

Volatility 2 examination starting points

Use the commands below as starting points. Replace <profile> with the profile you select from imageinfo and record any variation needed for the authorised lab environment. The commands guide your examination; the returned records provide the evidence.

Purpose	Starting command
Identify the image profile and capture time	`python2 vol.py -f memdumpWin7.mem imageinfo`
List active and residual processes	`python2 vol.py -f memdumpWin7.mem --profile=<profile> pslist` `python2 vol.py -f memdumpWin7.mem --profile=<profile> psscan`
Review network artefacts	`python2 vol.py -f memdumpWin7.mem --profile=<profile> netscan`
Review command history	`python2 vol.py -f memdumpWin7.mem --profile=<profile> cmdscan` `python2 vol.py -f memdumpWin7.mem --profile=<profile> consoles`

Locate USB and drive-mapping artefacts	`python2 vol.py -f memdumpWin7.mem --profile=<profile> hivelist`
	`python2 vol.py -f memdumpWin7.mem --profile=<profile> printkey -K "ControlSet001\Enum\USBSTOR"`
	`python2 vol.py -f memdumpWin7.mem --profile=<profile> printkey -K "Software\Microsoft\Windows\CurrentVersion\Explorer\MountPoints2"`

Review File Explorer history	`python2 vol.py -f memdumpWin7.mem --profile=<profile> shellbags --output=html --output-file=shellbags.html`

Evidence interpretation safeguards

- Validate a profile suggested by imageinfo with usable output from multiple relevant plugins and record the profile you selected.
- Treat network endpoints and process ownership as artefacts to corroborate. A closed connection or an unidentified PID needs careful interpretation.
- Assess a command-history entry with its process, timing, and related artefacts. A command string alone may not demonstrate that the copy operation completed.
- Use USBSTOR, MountedDevices, DeviceClasses, and MountPoints2 records to explain device, drive, and profile relationships. These records alone may not prove physical possession or a user identity at a particular time.
- Treat ShellBag times as folder-view or preference evidence unless other artefacts establish a different event.

Required report structure

1. Case reference and student identification. Include your full name, registration number, course code, and case title.
2. Executive conclusion. State the supported conclusion in a short, cautious paragraph and include a confidence level.
3. Evidence acquisition and integrity. Record the memory-image source, file name, size, hash, working-copy method, and tools used.
4. Method and profile selection. Explain the imageinfo result, selected profile, Volatility version, plugin commands, and time-zone treatment.
5. Findings. Answer each investigation question with Volatility output, registry paths, process IDs, offsets or records, timestamps, and evidence-log IDs.
6. Timeline. Present the material sequence in UTC and distinguish each observed artefact from your interpretation of its meaning.
7. Attribution assessment. Explain limitations, including memory-capture timing, profile versus human identity, command success, shared access, and incomplete or overwritten evidence.
8. Appendices. Include labelled screenshots, selected Volatility output, the completed timeline table, and the completed evidence log.

Evidence log template

```
kali@kali: ~/SBT_DF204_Case_Study_3
Session  Actions  Edit  View  Help

┌──(kali㉿kali)-[~]
└─$ mkdir -p ~/SBT_DF204_Case_Study_3/{original,working,outputs,screenshots}

┌──(kali㉿kali)-[~]
└─$ cd SBT_DF204_Case_Study_3

┌──(kali㉿kali)-[~/SBT_DF204_Case_Study_3]
└─$ stat -c '%s bytes' original/memdumpWin7.mem
1073676288 bytes

┌──(kali㉿kali)-[~/SBT_DF204_Case_Study_3]
└─$ sha256sum memdumpWin7.mem
sha256sum: memdumpWin7.mem: No such file or directory

┌──(kali㉿kali)-[~/SBT_DF204_Case_Study_3]
└─$ sha256sum original/memdumpWin7.mem
500bc1ba8b5ffa8053b6213d7e7aae0c84c18cce541dc53a55887e6e5099495e  original/memdumpWin7.mem

┌──(kali㉿kali)-[~/SBT_DF204_Case_Study_3]
└─$ cp original/memdumpWin7.mem working/memdumpWin7.mem

┌──(kali㉿kali)-[~/SBT_DF204_Case_Study_3]
└─$ sha256sum working/memdumpWin7.mem
500bc1ba8b5ffa8053b6213d7e7aae0c84c18cce541dc53a55887e6e5099495e  working/memdumpWin7.mem

┌──(kali㉿kali)-[~/SBT_DF204_Case_Study_3]
```

```
kali@kali: ~/SBT_DF204_Case_Study_3
Session  Actions  Edit  View  Help

┌──(kali㉿kali)-[~/SBT_DF204_Case_Study_3]
└─$ python2 ~/volatility/vol.py -f ~/SBT_DF204_Case_Study_3/working/memdumpWin7.mem imageinfo | tee ~/SBT_DF204_Case_Study
_3/outputs/01_imageinfo.txt
Volatility Foundation Volatility Framework 2.6.1
INFO    : volatility.debug    : Determining profile based on KDBG search...
** Failed to import volatility.plugins.registry.shutdown (ImportError: No module named Crypto.Hash)
** Failed to import volatility.plugins.getservicesids (ImportError: No module named Crypto.Hash)
** Failed to import volatility.plugins.timeliner (ImportError: No module named Crypto.Hash)
** Failed to import volatility.plugins.malware.apihooks (NameError: name 'distorm3' is not defined)
** Failed to import volatility.plugins.malware.servicediff (ImportError: No module named Crypto.Hash)
** Failed to import volatility.plugins.registry.userassist (ImportError: No module named Crypto.Hash)
** Failed to import volatility.plugins.getsids (ImportError: No module named Crypto.Hash)
** Failed to import volatility.plugins.registry.shellbags (ImportError: No module named Crypto.Hash)
** Failed to import volatility.plugins.evtlogs (ImportError: No module named Crypto.Hash)
** Failed to import volatility.plugins.registry.shimcache (ImportError: No module named Crypto.Hash)
** Failed to import volatility.plugins.tcaudit (ImportError: No module named Crypto.Hash)
** Failed to import volatility.plugins.registry.dumpregistry (ImportError: No module named Crypto.Hash)
** Failed to import volatility.plugins.registry.lsadump (ImportError: No module named Crypto.Hash)
** Failed to import volatility.plugins.malware.threads (NameError: name 'distorm3' is not defined)
** Failed to import volatility.plugins.mac.apihooks_kernel (ImportError: No module named distorm3)
** Failed to import volatility.plugins.registry.amcache (ImportError: No module named Crypto.Hash)
** Failed to import volatility.plugins.mac.check_syscall_shadow (ImportError: No module named distorm3)
** Failed to import volatility.plugins.malware.svcscan (ImportError: No module named Crypto.Hash)
** Failed to import volatility.plugins.registry.auditpol (ImportError: No module named Crypto.Hash)
** Failed to import volatility.plugins.ssdt (NameError: name 'distorm3' is not defined)
** Failed to import volatility.plugins.registry.registryapi (ImportError: No module named Crypto.Hash)
** Failed to import volatility.plugins.mac.apihooks (ImportError: No module named distorm3)
** Failed to import volatility.plugins.envars (ImportError: No module named Crypto.Hash)
          Suggested Profile(s) : Win7SP1x86_23418, Win7SP0x86, Win7SP1x86_24000, Win7SP1x86
                     AS Layer1 : IA32PagedMemoryPae (Kernel AS)
                     AS Layer2 : FileAddressSpace (/home/kali/SBT_DF204_Case_Study_3/working/memdumpWin7.mem)
                      PAE type : PAE
                           DTB : 0x185000L
                          KDBG : 0x82972c68L
          Number of Processors : 1
     Image Type (Service Pack) : 1
                KPCR for CPU 0 : 0x82973d00L
             KUSER_SHARED_DATA : 0xffdf0000L
```

Use an evidence log that lets an assessor locate the exact Volatility plugin, command, output, registry path, process record, or screenshot supporting each finding.

Evidence ID	Tool and plugin	Source record or path	Observed fact	Screenshot or appendix reference
E01	stat, sha256sum	original/memdumpWin7.mem and working/memdumpWin7.mem	Image is 1,073,676,288 bytes. SHA-256 of original and working copy are identical: 500bc1ba8b5ffa8053b6213d7e7aae0c84c18cce541dc53a55887e6e5099495e	
E02				
E03				
E04				
E05				
E06				